Express + Sequelize: алгоритм создания CRUD-сервера

Oct 7, 2026 · @Alex

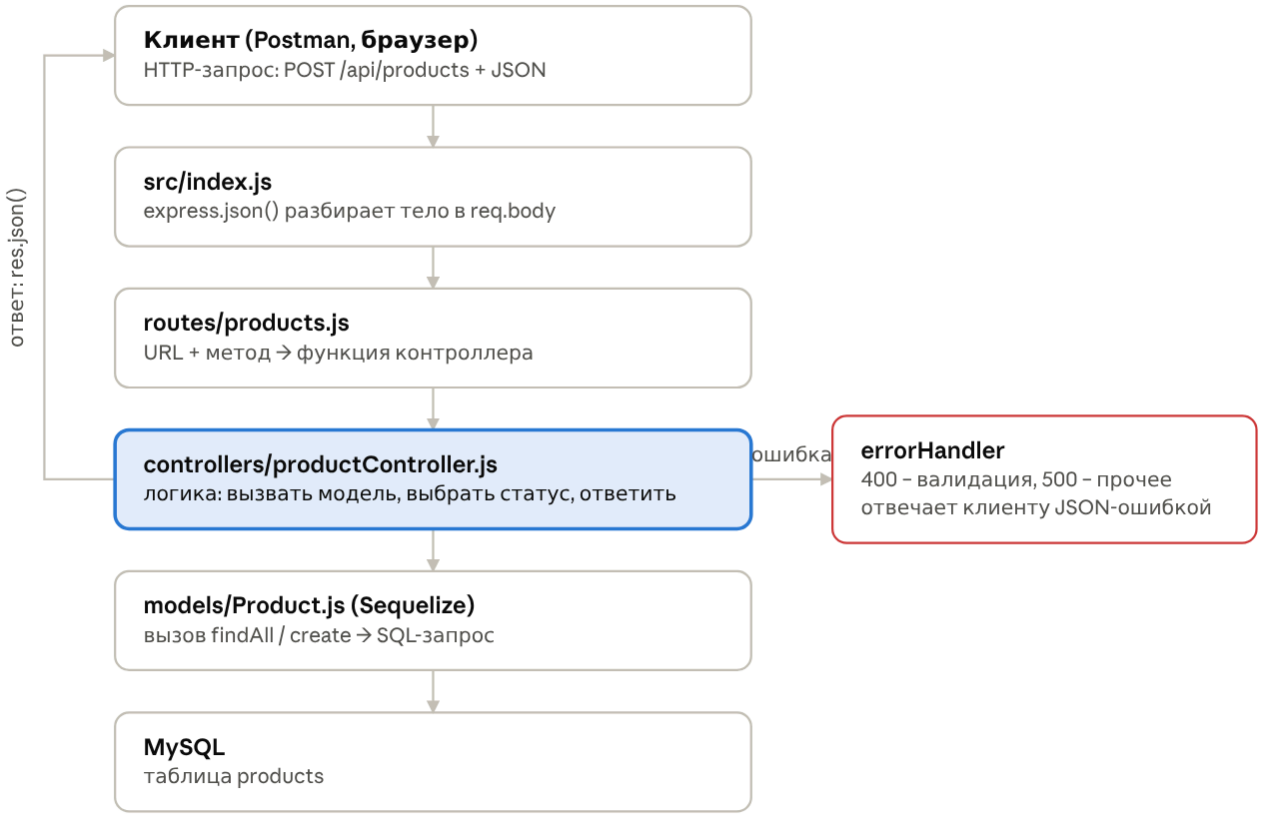
Обзор

В итоге получится REST API для сущности «Товар» (Product) с пятью операциями: получить список, получить один, создать, обновить, удалить. Сервер – Express, база – MySQL, работа с базой – через ORM Sequelize.

Главная идея порядка шагов: каждый следующий шаг опирается на предыдущий. Нельзя написать модель без подключения к базе, нельзя написать роут без контроллера, нельзя проверить CRUD без запущенного сервера. Если что-то сломалось – вернитесь к последнему шагу, где всё работало.

Итоговая структура проекта:

```
products-api/
├── .env                  ← пароли и настройки (НЕ в git)
├── .gitignore
├── package.json
└── src/
    ├── index.js          ← точка входа: запуск сервера
    ├── db.js             ← подключение Sequelize к базе
    ├── models/
    │   └── Product.js    ← описание таблицы
    ├── controllers/
    │   └── productController.js  ← логика CRUD
    ├── routes/
    │   └── products.js   ← URL → функция контроллера
    └── middlewares/
        └── errorHandler.js ← обработка ошибок
```



путь запроса через файлы проекта

Каждый файл отвечает за одно звено: роут не ходит в базу, модель не отвечает клиенту. Если запрос «теряется», ищите звено, после которого он перестал доходить.

Краткий алгоритм (подробности – в разделах ниже):

1. Создать проект и `package.json`.
2. Установить зависимости.
3. Поднять минимальный Express-сервер и убедиться, что он отвечает.
4. Создать базу данных и файл `.env`.
5. Настроить подключение Sequelize (`db.js`).
6. Проверить соединение с базой при старте сервера.
7. Описать модель (`models/Product.js`).
8. Синхронизировать модель с базой – создать таблицу.
9. Написать контроллер с пятью CRUD-функциями.
10. Подключить роуты к контроллеру и к приложению.
11. Добавить обработку ошибок.
12. Проверить все запросы через Postman / Thunder Client / curl.

Часть 1. Проект и Express-сервер

Шаг 1. Создать проект

Зачем: `package.json` хранит список зависимостей и команды запуска.

```
mkdir products-api
cd products-api
npm init -y
```

Сразу создайте `.gitignore`, чтобы не отправить в git лишнее:

```
node_modules
.env
```

Шаг 2. Установить зависимости

```
npm install express sequelize mysql2 dotenv
npm install -D nodemon
```

Пакет	Зачем нужен
express	веб-сервер и роутинг
sequelize	ORM: работа с таблицами через JS-объекты
mysql2	драйвер MySQL, без него Sequelize не подключится
dotenv	читает переменные из файла `.env`
nodemon (dev)	перезапускает сервер при сохранении файла

Для PostgreSQL вместо `mysql2` ставятся `pg pg-hstore`, всё остальное в алгоритме не меняется (кроме `dialect` и порта в шагах 4-5).

Включите ES-модули и добавьте скрипты в `package.json`. Строка `"type": "module"` разрешает писать `import` / `export` вместо `require` / `module.exports`:

```
"type": "module",
"scripts": {
  "start": "node src/index.js",
  "dev": "nodemon src/index.js"
}
```

Два правила ES-модулей, на которых чаще всего ошибаются:

- В импорте своих файлов расширение `.js` обязательно: `import sequelize from './db.js'`, а не `'./db'`.
- `export default` – один на файл, импортируется без фигурных скобок. Именованные `export const` импортируются в фигурных скобках: `import { getAll } from …`.

Шаг 3. Поднять минимальный сервер

Зачем: убедиться, что Express работает, ещё до того, как появится база. Так проще искать ошибки.

Файл `src/index.js`:

```
import express from 'express';

const app = express();
const PORT = 3000;

// без этой строки req.body будет undefined в POST/PUT
app.use(express.json());

app.get('/', (req, res) => {
  res.json({ status: 'ok' });
});

app.listen(PORT, () => {
  console.log(`Сервер запущен: http://localhost:${PORT}`);
});
```

Запуск: `npm run dev`. Откройте `http://localhost:3000` – должен прийти `{"status":"ok"}`.

Контрольная точка: сервер отвечает – переходим к базе. Не отвечает – дальше идти нет смысла.

Часть 2. Подключение к базе через Sequelize

Шаг 4. Создать базу и файл .env

Зачем: Sequelize создаёт таблицы, но саму базу данных – нет. А пароли нельзя писать прямо в коде.

Создайте базу (в MySQL Workbench, DBeaver или консоли mysql):

```
CREATE DATABASE products_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

`utf8mb4` нужен, чтобы кириллица и эмодзи сохранялись без «кракозябр».

Файл `.env` в корне проекта (рядом с `package.json`, не в `src`):

```
PORT=3000
DB_HOST=localhost
DB_PORT=3306
DB_NAME=products_db
DB_USER=root
DB_PASSWORD=ваш_пароль
```

Шаг 5. Настроить подключение (db.js)

Зачем: один объект `sequelize` на всё приложение. Его импортируют модели и `index.js`.

Файл `src/db.js`:

```
import { Sequelize } from 'sequelize';

const sequelize = new Sequelize(
  process.env.DB_NAME,
  process.env.DB_USER,
  process.env.DB_PASSWORD,
  {
    host: process.env.DB_HOST,
    port: process.env.DB_PORT,
    dialect: 'mysql',   // для PostgreSQL: 'postgres'
    logging: false,     // true – показывать SQL-запросы в консоли
  }
);

export default sequelize;
```

Шаг 6. Проверить соединение при старте

Зачем: сервер должен запускаться только тогда, когда база доступна. Иначе ошибка всплывёт позже, на первом запросе, и её будет труднее понять.

Обновите `src/index.js`. Важно: `import 'dotenv/config'` – самый первый импорт, до `db.js`. Импорты выполняются по порядку, и если `db.js` загрузится раньше, `process.env.DB_NAME` будет `undefined`.

```
import 'dotenv/config'; // ПЕРВЫЙ импорт!
import express from 'express';
import sequelize from './db.js';

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());

app.get('/', (req, res) => {
  res.json({ status: 'ok' });
});

const start = async () => {
  try {
    await sequelize.authenticate();
    console.log('Подключение к БД установлено');

    app.listen(PORT, () => {
      console.log(`Сервер запущен: http://localhost:${PORT}`);
    });
  } catch (err) {
    console.error('Не удалось подключиться к БД:', err.message);
    process.exit(1);
  }
};

start();
```

Контрольная точка: в консоли «Подключение к БД установлено». Если ошибка – проверьте `.env`, пароль и что сервер MySQL запущен.

Часть 3. Модель и таблица

Шаг 7. Описать модель

Зачем: модель – это JS-описание таблицы. Через неё мы будем делать все запросы к базе, не написав ни строки SQL.

Файл `src/models/Product.js`:

```
import { DataTypes } from 'sequelize';
import sequelize from '../db.js';

const Product = sequelize.define(
```

```
  'Product',
  {
    // id создаётся автоматически (INTEGER, PRIMARY KEY, AUTO_INCREMENT)
    name: {
      type: DataTypes.STRING,
      allowNull: false,
      validate: { notEmpty: true },
    },
    price: {
      type: DataTypes.INTEGER, // цена в целых единицах, например в сомах
      allowNull: false,
      validate: { min: 0 },
    },
    inStock: {
      type: DataTypes.BOOLEAN, // в MySQL хранится как TINYINT(1)
      defaultValue: true,
    },
  },
  {
    tableName: 'products', // имя таблицы в базе
    timestamps: true,      // поля createdAt и updatedAt
  }
);

export default Product;
```

Что здесь важно понять:

- `allowNull: false` – ограничение на уровне базы (NOT NULL).
- `validate` – проверка на уровне Sequelize, до отправки запроса в базу. При нарушении будет ошибка `SequelizeValidationError`.
- Если не указать `tableName`, Sequelize сам придумает имя во множественном числе (`Products`).

Шаг 8. Синхронизировать модель с базой

Зачем: `sync()` создаёт таблицу, если её ещё нет. Без этого шага первый же запрос упадёт с ошибкой «Table 'products_db.products' doesn't exist».

В `src/index.js` подключите модель и добавьте `sync()` после `authenticate()`:

```
import './models/Product.js'; // модель должна быть импортирована ДО sync()

// внутри start():
await sequelize.authenticate();
console.log('Подключение к БД установлено');

await sequelize.sync();
console.log('Модели синхронизированы');
```

Вариант	Что делает	Когда использовать
`sync()`	создаёт таблицу, если её нет; существующую не трогает	по умолчанию
`sync({ alter: true })`	подгоняет таблицу под изменённую модель	при разработке, если поменяли поля
`sync({ force: true })`	удаляет таблицу и создаёт заново – ВСЕ данные пропадут	только для учебных экспериментов

В реальных проектах вместо `sync()` используют миграции (`sequelize-cli`) – это следующая тема после CRUD.

Контрольная точка: в базе появилась таблица `products` с полями id, name, price, inStock, createdAt, updatedAt.

Часть 4. CRUD: контроллер, роуты, ошибки

Шаг 9. Написать контроллер

Зачем: контроллер содержит логику – что сделать с базой и что ответить клиенту. Роуты только решают, какую функцию вызвать.

Операция	HTTP-метод и URL	Метод Sequelize	Успешный статус
Получить все	GET /api/products	`findAll()`	200
Получить один	GET /api/products/:id	`findByPk(id)`	200 (или 404)
Создать	POST /api/products	`create(data)`	201
Обновить	PUT /api/products/:id	`findByPk` + `update(data)`	200 (или 404)

Операция	HTTP-метод и URL	Метод Sequelize	Успешный статус
Удалить	DELETE /api/products/:id	`destroy({ where })`	204 (или 404)

Файл `src/controllers/productController.js`:

```
import Product from '../models/Product.js';

// поля, которые клиент имеет право присылать
const ALLOWED_FIELDS = ['name', 'price', 'inStock'];

// GET /api/products
export const getAll = async (req, res) => {
  const products = await Product.findAll({ order: [['id', 'ASC']] });
  res.json(products);
};

// GET /api/products/:id
export const getOne = async (req, res) => {
  const product = await Product.findByPk(req.params.id);
  if (!product) {
    return res.status(404).json({ message: 'Товар не найден' });
  }
  res.json(product);
};

// POST /api/products
export const create = async (req, res) => {
  const product = await Product.create(req.body, { fields: ALLOWED_FIELDS });
  res.status(201).json(product);
};

// PUT /api/products/:id
export const update = async (req, res) => {
  const product = await Product.findByPk(req.params.id);
  if (!product) {
    return res.status(404).json({ message: 'Товар не найден' });
  }
  await product.update(req.body, { fields: ALLOWED_FIELDS });
  res.json(product);
};

// DELETE /api/products/:id
export const remove = async (req, res) => {
```

```
  const deletedCount = await Product.destroy({ where: { id: req.params.id }
});
  if (!deletedCount) {
    return res.status(404).json({ message: 'Товар не найден' });
  }
  res.status(204).end();
};
```

Здесь используются именованные экспорты (`export const`), потому что из одного файла экспортируется пять функций. Функция удаления названа `remove` , а не `delete` : `delete` – зарезервированное слово JavaScript.

Зачем `fields: ALLOWED_FIELDS` : без него клиент сможет прислать, например, `id` или `createdAt` и перезаписать их.

Обратите внимание на `return` перед ответом 404: без него код пойдёт дальше и попытается ответить второй раз – будет ошибка «Cannot set headers after they are sent».

Шаг 10. Подключить роуты

Зачем: связать URL и HTTP-метод с функцией контроллера.

Файл `src/routes/products.js` :

```
import { Router } from 'express';
import {
  getAll,
  getOne,
  create,
  update,
  remove,
} from '../controllers/productController.js';

const router = Router();

router.get('/', getAll);
router.get('/:id', getOne);
router.post('/', create);
router.put('/:id', update);
router.delete('/:id', remove);

export default router;
```

В `src/index.js` после `app.use(express.json())` :

```
import productRoutes from './routes/products.js'; // вверху файла, к
остальным импортам

app.use('/api/products', productRoutes);
```

Префикс `/api/products` задаётся один раз здесь, поэтому внутри роутера пути короткие: `/` и `/:id`.

Шаг 11. Добавить обработку ошибок

Зачем: если база вернёт ошибку (например, не прошла валидация), клиент должен получить понятный ответ, а сервер – не упасть.

Файл `src/middlewares/errorHandler.js`:

```
// 404 – ни один роут не подошёл
export const notFound = (req, res) => {
  res.status(404).json({ message: `Маршрут ${req.method} ${req.originalUrl}
не найден` });
};

// обработчик ошибок – у него ВСЕГДА 4 параметра
export const errorHandler = (err, req, res, next) => {
  if (err.name === 'SequelizeValidationError') {
    return res.status(400).json({
      message: 'Ошибка валидации',
      errors: err.errors.map((e) => e.message),
    });
  }

  console.error(err);
  res.status(500).json({ message: 'Внутренняя ошибка сервера' });
};
```

Подключение в `src/index.js` – строго ПОСЛЕ всех роутов:

```
import { notFound, errorHandler } from './middlewares/errorHandler.js';

app.use('/api/products', productRoutes);
app.use(notFound);     // после роутов
app.use(errorHandler); // самым последним
```

Про async-ошибки: в Express 5 (его ставит `npm install express` по умолчанию) ошибка из async-функции сама попадает в `errorHandler`. В Express 4 так не работает – там каждую функцию контроллера нужно обернуть в `try { … }`

`catch (err) { next(err); }`. Версию смотрите в `package.json`.

Итоговый `src/index.js` целиком:

```
import 'dotenv/config';
import express from 'express';
import sequelize from './db.js';
import './models/Product.js';
import productRoutes from './routes/products.js';
import { notFound, errorHandler } from './middlewares/errorHandler.js';

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());

app.get('/', (req, res) => res.json({ status: 'ok' }));
app.use('/api/products', productRoutes);

app.use(notFound);
app.use(errorHandler);

const start = async () => {
  try {
    await sequelize.authenticate();
    console.log('Подключение к БД установлено');
    await sequelize.sync();
    console.log('Модели синхронизированы');

    app.listen(PORT, () => {
      console.log(`Сервер запущен: http://localhost:${PORT}`);
    });
  } catch (err) {
    console.error('Не удалось запустить сервер:', err.message);
    process.exit(1);
  }
};

start();
```

Часть 5. Проверка и частые ошибки

Шаг 12. Проверить все операции

Проверяйте в этом порядке – так у каждого следующего запроса уже есть данные. В Postman / Thunder Client для POST и PUT выберите Body → raw → JSON.

№	Запрос	Тело запроса	Ожидаемый ответ
1	POST /api/products	`{"name":"Шлем","price":4500}`	201, объект с id
2	GET /api/products	–	200, массив из одного товара
3	GET /api/products/1	–	200, этот товар
4	PUT /api/products/1	`{"price":4000}`	200, цена изменилась
5	DELETE /api/products/1	–	204, пустой ответ
6	GET /api/products/1	–	404
7	POST /api/products	`{"name":"","price":-5}`	400, ошибка валидации

То же через curl:

```
curl -X POST http://localhost:3000/api/products \
  -H "Content-Type: application/json" \
  -d '{"name":"Шлем","price":4500}'

curl http://localhost:3000/api/products

curl -X PUT http://localhost:3000/api/products/1 \
  -H "Content-Type: application/json" \
  -d '{"price":4000}'

curl -X DELETE http://localhost:3000/api/products/1
```

Частые ошибки

Симптом	Причина	Решение
`Cannot use import statement outside a module`	нет `"type": "module"` в `package.json`	шаг 2: добавить строку
`ERR_MODULE_NOT_FOUND`	в импорте своего файла нет `.js`	`'./db.js'`, а не `'./db'`
`does not provide an export named …`	перепутаны default и именованный экспорт	`export default` – без скобок, `export const` – в `{ }`
`req.body` – undefined	нет `app.use(express.json())` или он ниже роутов	поставить сразу после создания `app`
`req.body` пустой `{}`	не указан заголовок `Content-Type: application/json`	в Postman выбрать raw → JSON
`Access denied for user 'root'@'localhost'`	неверный пароль или `.env` не прочитан	проверить `.env`; `import 'dotenv/config'` – первым
`Unknown database 'products_db'`	база не создана	шаг 4: `CREATE DATABASE`
`Table 'products_db.products' doesn't exist`	таблица не создана	шаг 8: импорт модели и `sync()`
`Please install mysql2 package manually`	не установлен драйвер	`npm install mysql2`
`connect ECONNREFUSED 127.0.0.1:3306`	сервер MySQL не запущен	запустить службу MySQL
`Cannot set headers after they are sent`	два ответа в одной функции	`return res.status(404)…`
Любой путь даёт 404	`notFound` подключён выше роутов	`notFound` и `errorHandler` – в самом конце

Симптом	Причина	Решение
`EADDRINUSE: port 3000`	сервер уже запущен в другом терминале	закрыть старый процесс или сменить PORT

Чек-лист перед сдачей

- [ ] `npm run dev` запускает сервер без ошибок
- [ ] В консоли есть «Подключение к БД установлено»
- [ ] `.env` и `node_modules` в `.gitignore`, пароль не попал в репозиторий
- [ ] Все 5 операций работают и возвращают правильные статусы (200, 201, 204, 404)
- [ ] Невалидные данные возвращают 400, а не 500
- [ ] Код разделён по папкам: models, controllers, routes, middlewares